EXPERIMENT 10

Design and Implementation of an End-to-End DevOps Pipeline Using a Mini Project

Student Name: _____

Roll Number: _____

Class / Batch: _____

Department: _____

College: _____

Academic Year: _____

Date: _____

1. Aim

To design and implement a mini project demonstrating a DevOps workflow using Git, GitHub, GitHub Actions, automated testing, Docker containerization, and application execution.

2. Objectives

• Use Git and GitHub for source-code version control.

• Configure an automated CI workflow using GitHub Actions.

• Execute automated tests to verify application functionality.

• Package the application in a Docker image.

• Run the containerized application and document the results.

3. Problem Statement

Student work often involves multiple tasks and deadlines. The Student Task Manager provides a simple web interface for adding tasks, marking them complete, and deleting them. The application is used as the software component for demonstrating an end-to-end DevOps workflow.

4. Tools and Technologies

Tool / Technology	Purpose
Python 3.12	Application programming language
Flask	Web framework
VS Code	Code editing
Git	Version control
GitHub	Remote repository and collaboration
GitHub Actions	Continuous integration workflow
pytest	Automated testing
Docker	Containerization and local execution

5. System Architecture

Developer edits code locally → Git commit and push → GitHub repository → GitHub Actions checks out code → installs dependencies → runs pytest → builds Docker image. The image can then be run locally as a container and accessed through a web browser.

Pipeline stages: Code → Version Control → Build/Dependency Setup → Automated Test → Docker Image Build → Container Run / Verification.

6. Application Features

• Add a task using a web form.

• View the current task list.

• Mark a task complete or undo completion.

• Delete a task.

• Check service status through the /health endpoint.

Tasks are held in memory for simplicity; they are not persisted after the process restarts.

7. Implementation

7.1 Version Control

The project is stored in a Git repository and uploaded to GitHub. Commits record changes and provide a history of development.

```
git init
git add .
git commit -m "Initial Student Task Manager project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/student-task-manager-devops.git
git push -u origin main
```

Replace the example repository URL with the URL of the repository you create.

7.2 Automated Testing

The test_app.py file uses pytest and Flask's test client to verify the home page, task creation, blank-input handling, task completion, task deletion, and health endpoint.

```
pip install -r requirements-dev.txt
pytest -v
```

Actual test result: _____

[Insert screenshot: pytest-results.png]

7.3 Continuous Integration with GitHub Actions

The workflow file .github/workflows/ci.yml runs on pushes and pull requests to main or master, and can also be started manually. It checks out the code, sets up Python, installs dependencies, runs pytest, and builds a Docker image.

Actual GitHub Actions result: _____

[Insert screenshot: github-actions-success.png]

7.4 Docker Containerization

The Dockerfile uses a lightweight Python base image, installs application dependencies, copies the Flask application and template, exposes port 5000, and starts the application as a non-root user.

```
docker build -t student-task-manager .
docker run --rm -p 5000:5000 student-task-manager
```

Open http://localhost:5000 in a browser. Check service status at http://localhost:5000/health.

Actual Docker build/run result: _____

[Insert screenshots: docker-build.png and app-running.png]

7.5 Deployment

For this demonstration, deployment means running the built Docker image as a local container. The application is accessed through the host's port 5000. This report does not claim external cloud deployment or automated image publishing.

Deployment verification: _____

8. Results and Observations

Complete this section only after executing the project:

• Application opened successfully: Yes / No

• Automated tests passed: Yes / No; count: _____

• GitHub Actions workflow succeeded: Yes / No

• Docker image built successfully: Yes / No

• Containerized application accessible: Yes / No

The workflow is designed to provide repeatable checks and detect test failures before the Docker image build stage completes.

9. Advantages

• Version history makes changes traceable.

• Automated testing reduces repetitive manual checks.

• Docker packages the runtime environment consistently.

• GitHub Actions provides visible pipeline logs.

• The workflow can be rerun after each code update.

10. Limitations

• Task data is stored in memory and resets on restart.

• The CI workflow builds a Docker image but does not publish it to a container registry.

• The demonstrated deployment is local, not a public cloud deployment.

• GitHub Actions results depend on successfully pushing the repository and running the workflow.

11. Conclusion

The Student Task Manager mini project provides a practical demonstration of version control, automated testing, a GitHub Actions CI workflow, Docker image creation, and container-based application execution. After the commands and workflow are run and verified, screenshots and actual results should be added to this report before submission.

12. References

• Git documentation: https://git-scm.com/doc

• GitHub Actions documentation: https://docs.github.com/actions

• Docker documentation: https://docs.docker.com/

• Flask documentation: https://flask.palletsprojects.com/

• pytest documentation: https://docs.pytest.org/